A step-by-step procedure for deploying a Cloud Native Monitoring Application on Kubernetes.

Prerequisites:

1) AWS Account

Follow below official site to create aws account
https://docs.aws.amazon.com/accounts/latest/reference/manage-acct-creating.html

2) GITHUB Account

To create github account:

- ❒ **Login to https://github.com/**
- ❒ **Provide username, email and password**
- ❒ **Verify account**
- ❒ **Click on join a free plan**
- ❒ **You will get some questions, fill it**
- ❒ **Click on complete setup**
- ❒ **Go to Gmail and verify your account**

3) Python3 Installed

Follow python official website to install (https://www.python.org/downloads/)

4)Docker Installed

Follow docker official website to install (https://docs.docker.com/get-docker/)

Installation steps:

```
sudo apt-get update -y
sudo apt-get install docker.io -y
sudo usermod -aG docker $USER  # Replace with your system's username, e.g., 'ubuntu'
newgrp docker
sudo chmod 777 /var/run/docker.sock
sudo systemctl restart docker
sudo systemctl status docker
```

5)Kubectl Installed

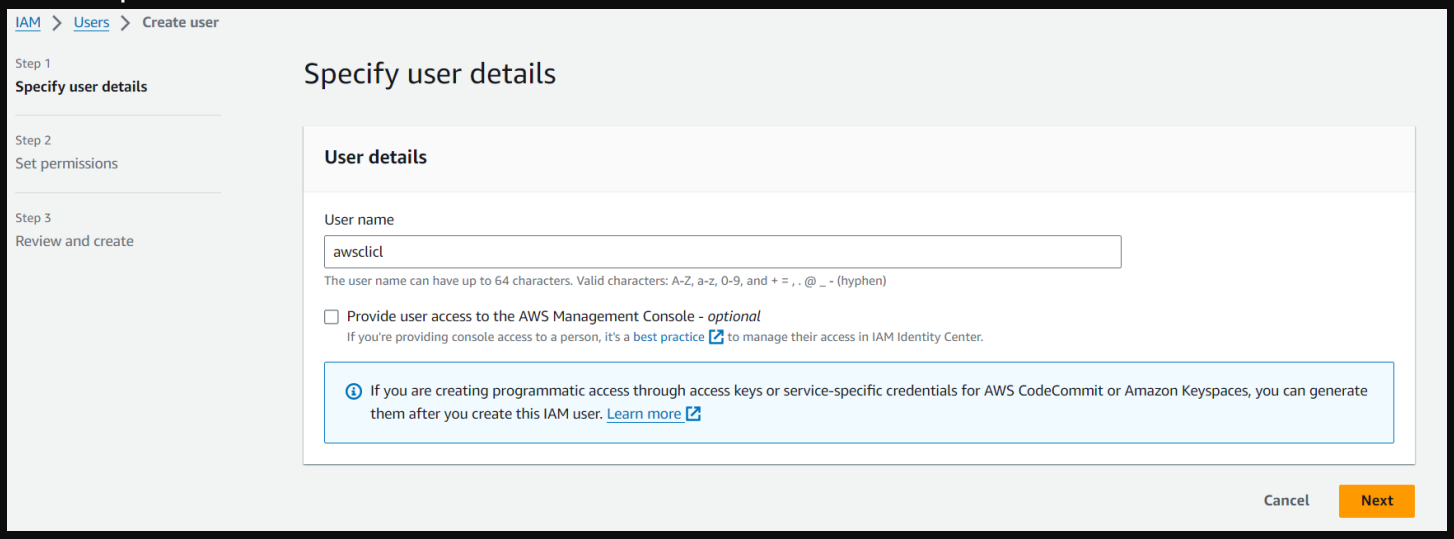
https://kubernetes.io/docs/tasks/tools/

Installation steps:

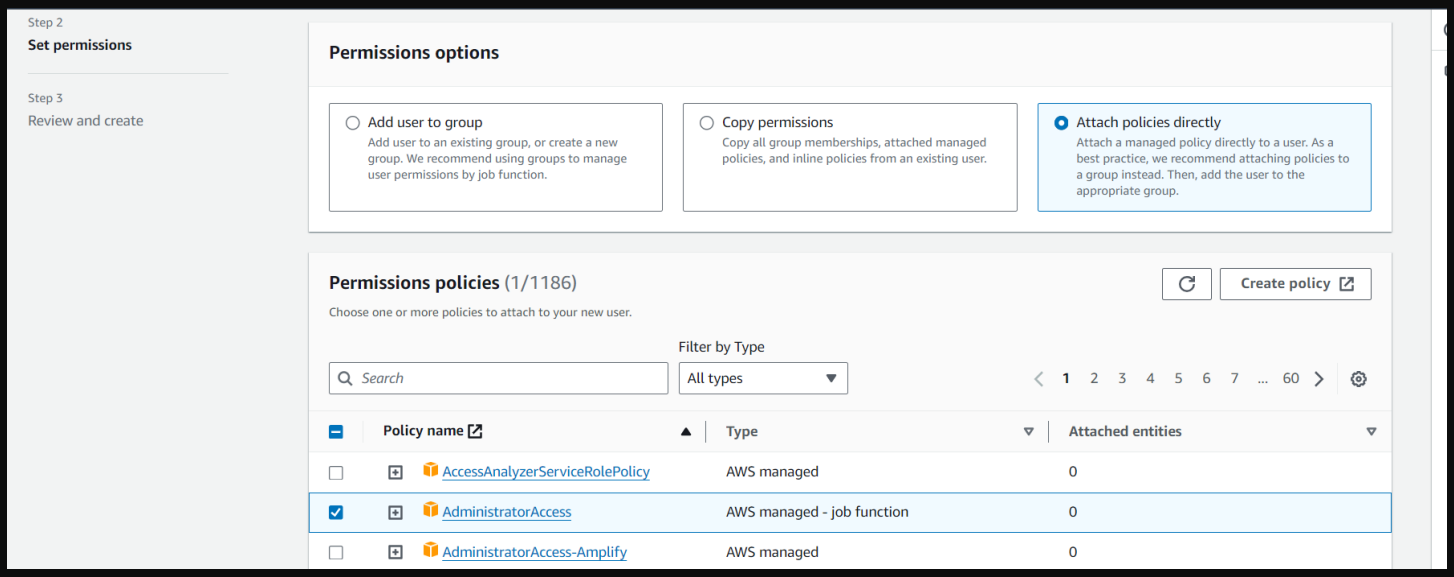
```
curl -LO https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
chmod +x ./kubectl
sudo mv ./kubectl /usr/local/bin/kubectl
kubectl version --client
```

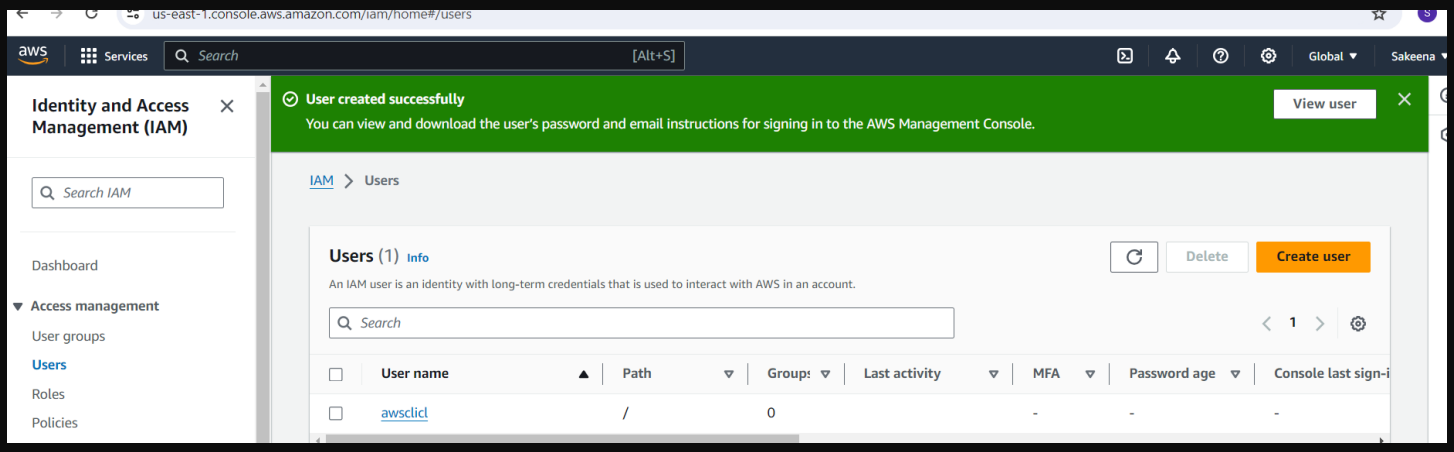
6)AWS configured with CLI

Navigate to the IAM dashboard on AWS, then select "Users." Enter the username and proceed to the next step

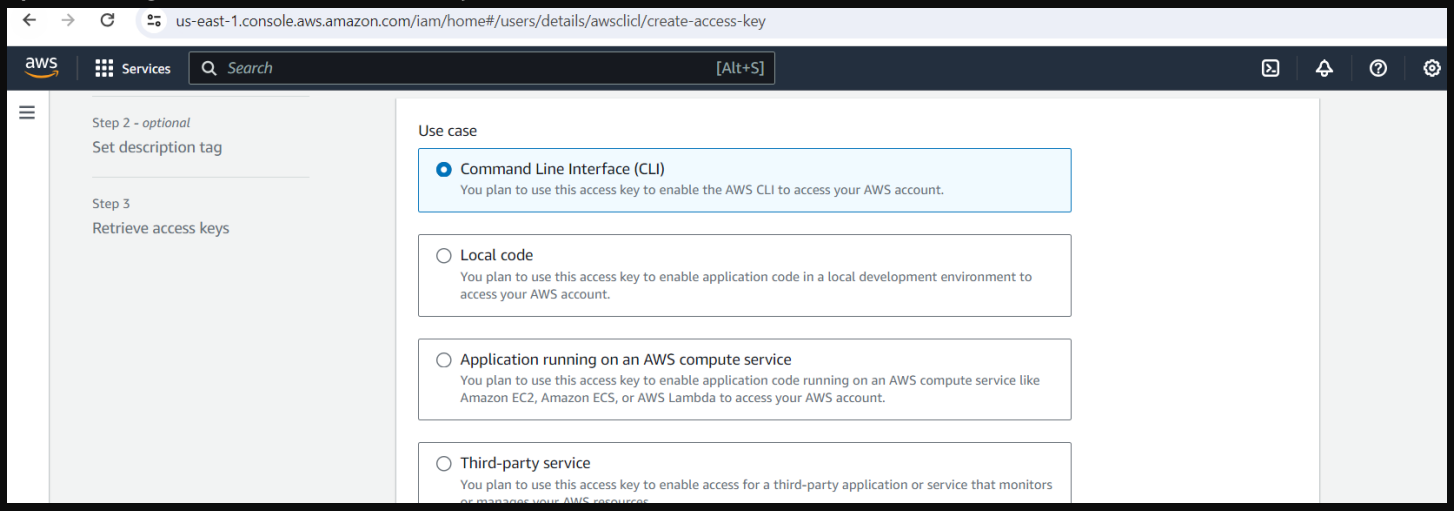


Assign permissions by attaching policies directly, opting for "Administrator access," and then create the user

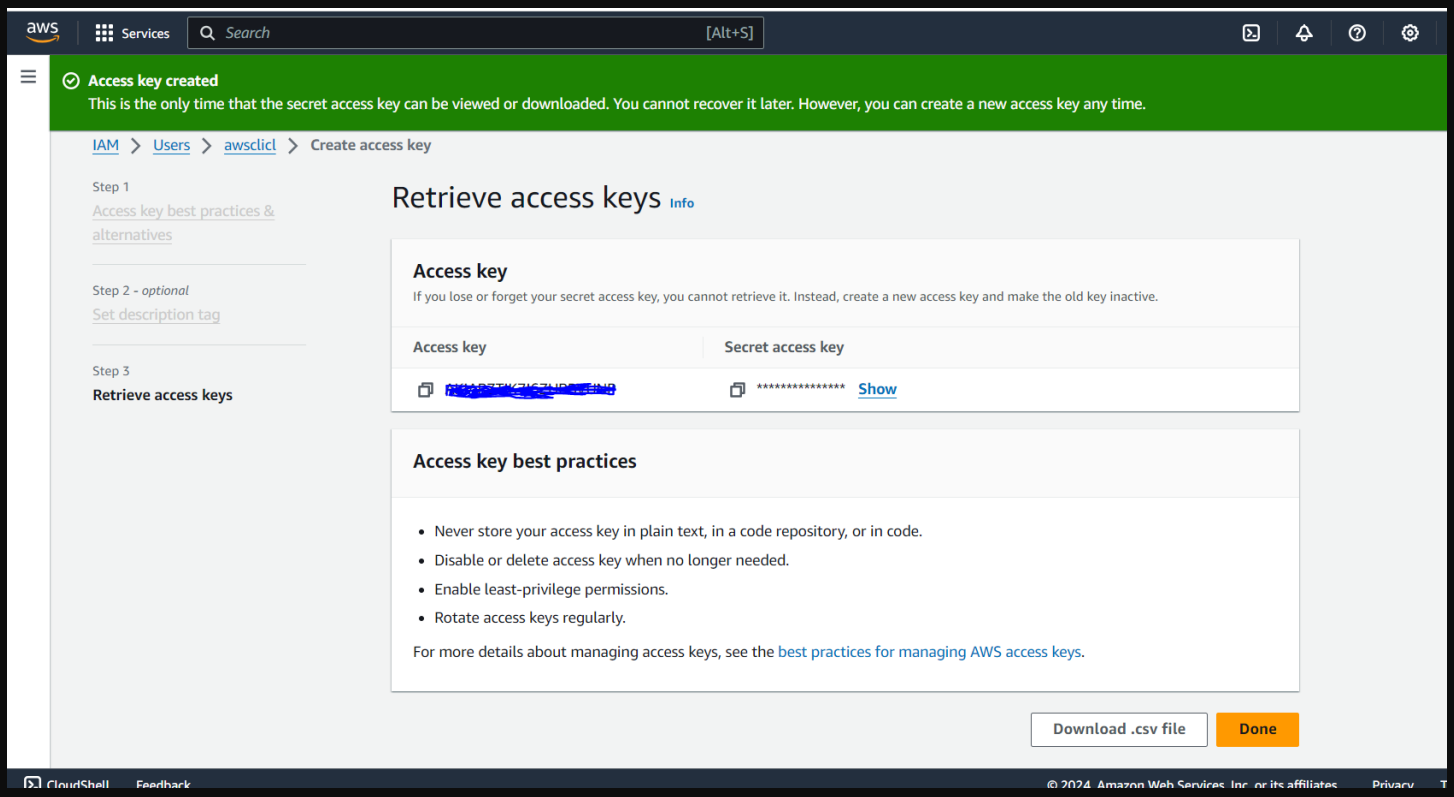




Within the user settings, locate "Create access key," and choose the command line interface (CLI) option to generate an access key.



Upon creation, you can view or download the access key and secret access key either from the console or via CSV download.



Now go to your terminal and follow below steps:

```
sudo apt install unzip
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws configure (input created accesskeyid and secret access key)
cat ~/.aws/config
cat ~/.aws/credentials
aws iam list-users (to list all IAM users in an AWS account)
```

7) Code editor (Vscode)

Utilize Visual Studio Code or any other platform that supports python application execution according to your preference (https://code.visualstudio.com/)

Lets start the project:

Step1: Make sure to have all above prerequisites installed

```
sakeena@ubuntu: ~
sakeena@ubuntu:~$ python3 --version
Python 3.10.12
sakeena@ubuntu:~$ docker --version
Docker version 26.0.0, build 2ae903e
sakeena@ubuntu:~$ kubectl version --client
Client Version: v1.29.3
Kustomize Version: v5.0.4-0.20230601165947-6ce0bf390ce3
sakeena@ubuntu:~$ aws iam list-users
{
    "Users": [
        {
            "Path": "/",
            "UserName": "awsclicl",
            "UserId": "AIDARZTIK7I6WZGON2BYV",
            "Arn": "arn:aws:iam::123699132989:user/awsclicl",
            "CreateDate": "2024-04-14T04:10:16+00:00"
        }
    ]
}
sakeena@ubuntu:~$
```

Step2:
Checkout the code present on below github repo by nasiullah choudhari (cloud champ - creator of this project)https://github.com/N4si/cloud-native-monitoring-app
Now go to your terminal and clone the code in to your local machine

```
sakeena@ubuntu: ~/cloud-native-monitoring-app
sakeena@ubuntu:~$ git clone https://github.com/Sakeena19/cloud-native-monitoring-app.git
Cloning into 'cloud-native-monitoring-app'...
remote: Enumerating objects: 28, done.
remote: Counting objects: 100% (16/16), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 28 (delta 9), reused 4 (delta 3), pack-reused 12
Receiving objects: 100% (28/28), 8.06 KiB | 275.00 KiB/s, done.
Resolving deltas: 100% (9/9), done.
sakeena@ubuntu:~$ ls
aws                         Desktop    kubectl         Music     projects  Templates
awscliv2.zip                Documents  kubectl.sha256  mydir     Public    Videos
cloud-native-monitoring-app Downloads  linux           Pictures  snap
sakeena@ubuntu:~$ cd cloud-native-monitoring-app/
sakeena@ubuntu:~/cloud-native-monitoring-app$ ls
app.py  Dockerfile  ecr.py  eks.py  README.md  requirements.txt  templates
sakeena@ubuntu:~/cloud-native-monitoring-app$
```

The app directory contain below files which can be used as follows:

- app.py: This Python script utilizes the psutil library to monitor CPU and memory usage. It creates a Flask web app that displays these metrics. If either metric exceeds 80%, it prompts to scale up resources.
- requirements.txt: contains all necessary dependencies for this project
- Dockerfile: This Dockerfile sets up a Python environment based on Python 3.9 on Debian Buster. It installs the required dependencies specified in requirements.txt using pip3. Then, it exposes port 5000 and configures Flask to run on all network interfaces. Finally, it starts the Flask application using flask run.
- ecr.py: This Python script utilizes the Boto3 library to interact with the Amazon Elastic Container Registry (ECR). It creates a new repository named "my_monitoring_app_image" using the create_repository method of the ECR client. It then retrieves the URI of the newly created repository and prints it to the console. Essentially, this code automates the process of creating an ECR repository for storing container images.
- eks.py: This script automates Flask application deployment to Kubernetes by creating a Deployment and Service. It defines a Deployment for the Flask app and a Service to expose it.

Step3:

To install required dependencies using pip:
pip3 install psutil (a python library which retrieves info about system utilization)
pip3 install -r requirements.txt

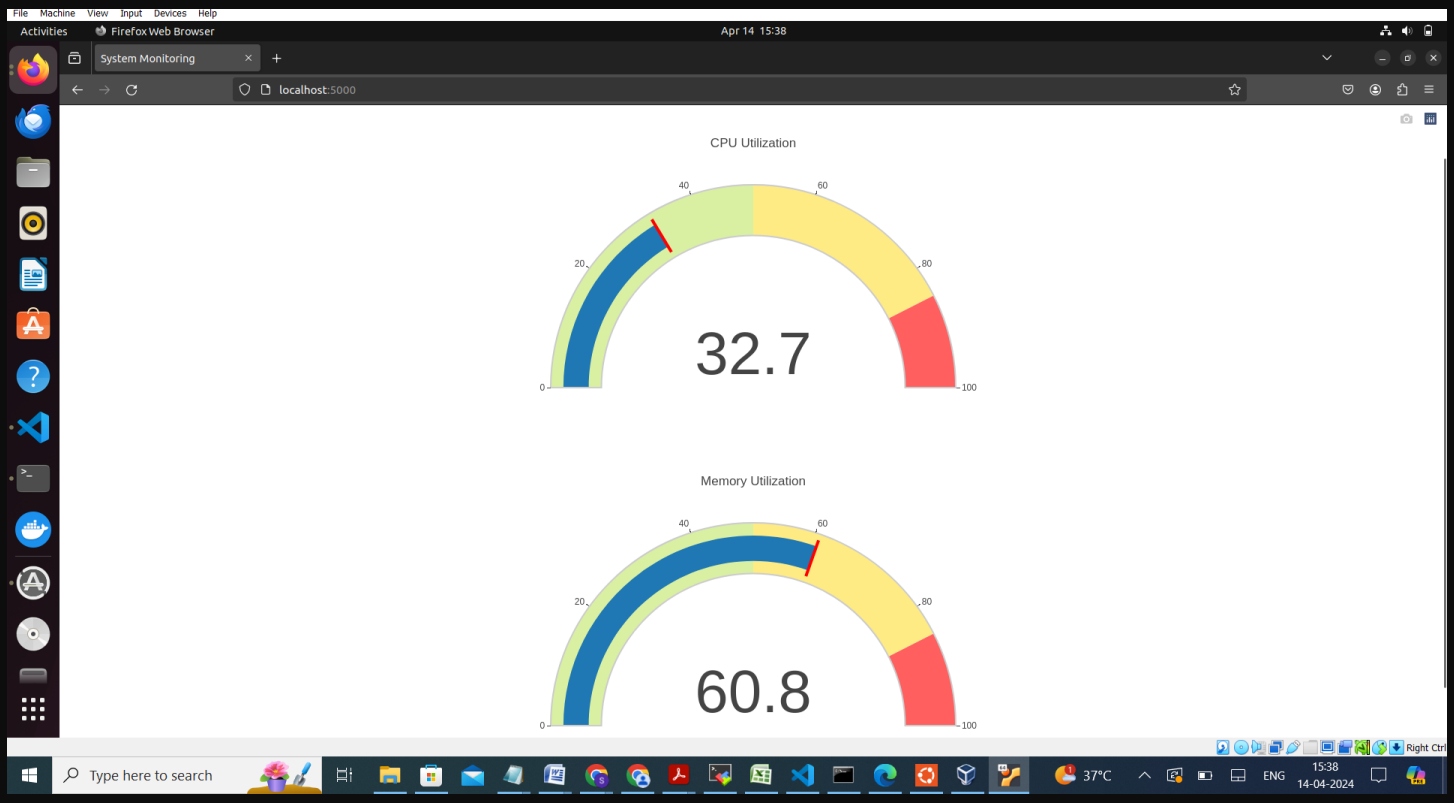
```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                                    bash

sakeena@ubuntu:~/cloud-native-monitoring-app$ pip3 install psutil
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: psutil in /home/sakeena/.local/lib/python3.10/site-packages (5.8.0)
sakeena@ubuntu:~/cloud-native-monitoring-app$ pip3 install -r requirements.txt
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: Flask==2.2.3 in /home/sakeena/.local/lib/python3.10/site-packages (from -r requirements.txt (line 1)) (2.2.3)
Requirement already satisfied: MarkupSafe==2.1.2 in /home/sakeena/.local/lib/python3.10/site-packages (from -r requirements.txt (line 2)) (2.1.2)
Requirement already satisfied: Werkzeug==2.2.3 in /home/sakeena/.local/lib/python3.10/site-packages (from -r requirements.txt (line 3)) (2.2.3)
Requirement already satisfied: itsdangerous==2.1.2 in /home/sakeena/.local/lib/python3.10/site-packages (from -r requirements.txt (line 4)) (2.1.2)
Requirement already satisfied: psutil==5.8.0 in /home/sakeena/.local/lib/python3.10/site-packages (from -r requirements.txt (line 5)) (5.8.0)
Requirement already satisfied: plotly==5.5.0 in /home/sakeena/.local/lib/python3.10/site-packages (from -r requirements.txt (line 6)) (5.5.0)
Requirement already satisfied: tenacity==8.0.1 in /home/sakeena/.local/lib/python3.10/site-packages (from -r requirements.txt (line 7)) (8.0.1)
Requirement already satisfied: boto3==1.9.148 in /home/sakeena/.local/lib/python3.10/site-packages (from -r requirements.txt (line 8)) (1.9.148)
Requirement already satisfied: kubernetes==10.0.1 in /home/sakeena/.local/lib/python3.10/site-packages (from -r requirements.txt (line 9)) (10.0.1)
Requirement already satisfied: Jinja2>=3.0 in /home/sakeena/.local/lib/python3.10/site-packages (from Flask==2.2.3->-r requirements.txt (line 1)) (3.1.3)
Requirement already satisfied: click>=8.0 in /usr/lib/python3/dist-packages (from Flask==2.2.3->-r requirements.txt (line 1)) (8.0.3)
Requirement already satisfied: six in /usr/lib/python3/dist-packages (from plotly==5.5.0->-r requirements.txt (line 6)) (1.16.0)
Requirement already satisfied: jmespath<1.0.0,>=0.7.1 in /home/sakeena/.local/lib/python3.10/site-packages (from boto3==1.9.148->-r requirements.txt (line 8)) (0.10.0)
Requirement already satisfied: s3transfer<0.3.0,>=0.2.0 in /home/sakeena/.local/lib/python3.10/site-packages (from boto3==1.9.148->-r requirements.txt (line 8)) (0.2.1)
Requirement already satisfied: botocore<1.13.0,>=1.12.148 in /home/sakeena/.local/lib/python3.10/site-packages (from boto3==1.9.148->-r requirements.txt (line 8)) (1.12.253)
Requirement already satisfied: google-auth>=1.0.1 in /home/sakeena/.local/lib/python3.10/site-packages (from kubernetes==10.0.1->-r requirements.txt (line 9)) (2.29.0)
Requirement already satisfied: pyyaml>=3.12 in /usr/lib/python3/dist-packages (from kubernetes==10.0.1->-r requirements.txt (line 9)) (5.4.1)
Requirement already satisfied: setuptools>=21.0.0 in /usr/lib/python3/dist-packages (from kubernetes==10.0.1->-r requirements.txt (line 9)) (59.6.0)
Requirement already satisfied: requests-oauthlib in /home/sakeena/.local/lib/python3.10/site-packages (from kubernetes==10.0.1->-r requirements.txt (line 9)) (2.0.0)
Requirement already satisfied: python-dateutil>=2.5.3 in /usr/lib/python3/dist-packages (from kubernetes==10.0.1->-r requirements.txt (line 9)) (2.8.1)
Requirement already satisfied: requests in /usr/lib/python3/dist-packages (from kubernetes==10.0.1->-r requirements.txt (line 9)) (2.25.1)
```

step4: Run python3 app.py and check if the app is accessible in UI.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

sakeena@ubuntu:~/cloud-native-monitoring-app$ python3 app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.0.2.15:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 142-032-798
```



This is how we can run this monitoring application in localhost

Lets containerize this application using docker:

step5: write Dockerfile (present in git repo) and create an image from it
command: docker build -t my-flask-app . (-t = tag image with specific name, . = current directory)

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

sakeena@ubuntu:~/cloud-native-monitoring-app$ docker build -t my-flask-app .
[+] Building 24.0s (5/10)                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                0.1s
 => => transferring dockerfile: 232B                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim-buster                           3.6s
 => [auth] library/python:pull token for registry-1.docker.io                                       0.0s
 => [internal] load .dockerignore                                                                   0.3s
 => => transferring context: 2B                                                                     0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim-buster@sha256:320a7a4250aba4249f458872adecf92eea88dc6abd2d76dc5c0f01cac9b53990   19.8s
 => => resolve docker.io/library/python:3.9-slim-buster@sha256:320a7a4250aba4249f458872adecf92eea88dc6abd2d76dc5c0f01cac9b53990    0.3s
 => => sha256:320a7a4250aba4249f458872adecf92eea88dc6abd2d76dc5c0f01cac9b53990 988B / 988B          0.0s
 => => sha256:d5cca64dca485c37ccf06721e36a93bf4331b0404bfd3ef2c7873867965359b7 1.37kB / 1.37kB      0.0s
 => => sha256:c84dbfe3b8deeb39e17d121220107f8354a9083b468a320a77708cd128f11c87 6.82kB / 6.82kB      0.0s
 => => sha256:8b91b88d557765cd8c6802668755a3f6dc4337b6ce15a17e4857139e5fc964f3 27.14MB / 27.14MB    4.5s
 => => sha256:824416e234237961c9c5d4f41dfe5b295a3c35a671ee52889bfb08d8e257ec4c 2.78MB / 2.78MB      1.6s
 => => sha256:8d53da26040835f622504d7762fad14d226ac414efeb5363f5febebc89ff224d 11.04MB / 11.04MB    3.1s
 => => sha256:84c8c79126f669beec1dcf6f34cd88094471745570c19c29b465dfa7db1fdabd 243B / 243B          2.5s
 => => sha256:2e1c130fa3ec1777a82123374b4c500623959f903c1dd731ee4a83e1f1b38ff2 3.14MB / 3.14MB      4.6s
 => => extracting sha256:8b91b88d557765cd8c6802668755a3f6dc4337b6ce15a17e4857139e5fc964f3           14.7s
 => [internal] load build context                                                                   0.7s
 => => transferring context: 49.01kB                                                                0.3s
```

To check the images created use : docker images

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

 => => sha256:84c8c79126f669beec1dcf6f34cd88094471745570c19c29b465dfa7db1fdabd 243B / 243B
 => => sha256:2e1c130fa3ec1777a82123374b4c500623959f903c1dd731ee4a83e1f1b38ff2 3.14MB / 3.14MB
 => => extracting sha256:8b91b88d557765cd8c6802668755a3f6dc4337b6ce15a17e4857139e5fc964f3
 => => extracting sha256:824416e234237961c9c5d4f41dfe5b295a3c35a671ee52889bfb08d8e257ec4c
 => => extracting sha256:8d53da26040835f622504d7762fad14d226ac414efeb5363f5febebc89ff224d
 => => extracting sha256:84c8c79126f669beec1dcf6f34cd88094471745570c19c29b465dfa7db1fdabd
 => => extracting sha256:2e1c130fa3ec1777a82123374b4c500623959f903c1dd731ee4a83e1f1b38ff2
 => [internal] load build context
 => => transferring context: 49.01kB
 => [2/5] WORKDIR /app
 => [3/5] COPY requirements.txt .
 => [4/5] RUN pip3 install --no-cache-dir -r requirements.txt
 => [5/5] COPY . .
 => exporting to image
 => => exporting layers
 => => writing image sha256:d39bbeb4f1e8bd35fb5fe8954223600880701d58ccbd9679a848ed45a530b477
 => => naming to docker.io/library/my-flask-app

Reduce build time with Docker Build Cloud: https://docs.docker.com/go/docker-build-cloud
sakeena@ubuntu:~/cloud-native-monitoring-app$ docker images
REPOSITORY          TAG       IMAGE ID       CREATED          SIZE
my-flask-app        latest    d39bbeb4f1e8   51 seconds ago   327MB
mytodoapp           latest    8635e2cdff64   3 weeks ago      266MB
nasi101/mytodoapp   latest    1829ccb7f0f1   3 weeks ago      243MB
hello-world         latest    d2c94e258dcb   11 months ago    13.3kB
sakeena@ubuntu:~/cloud-native-monitoring-app$
```

Step6:

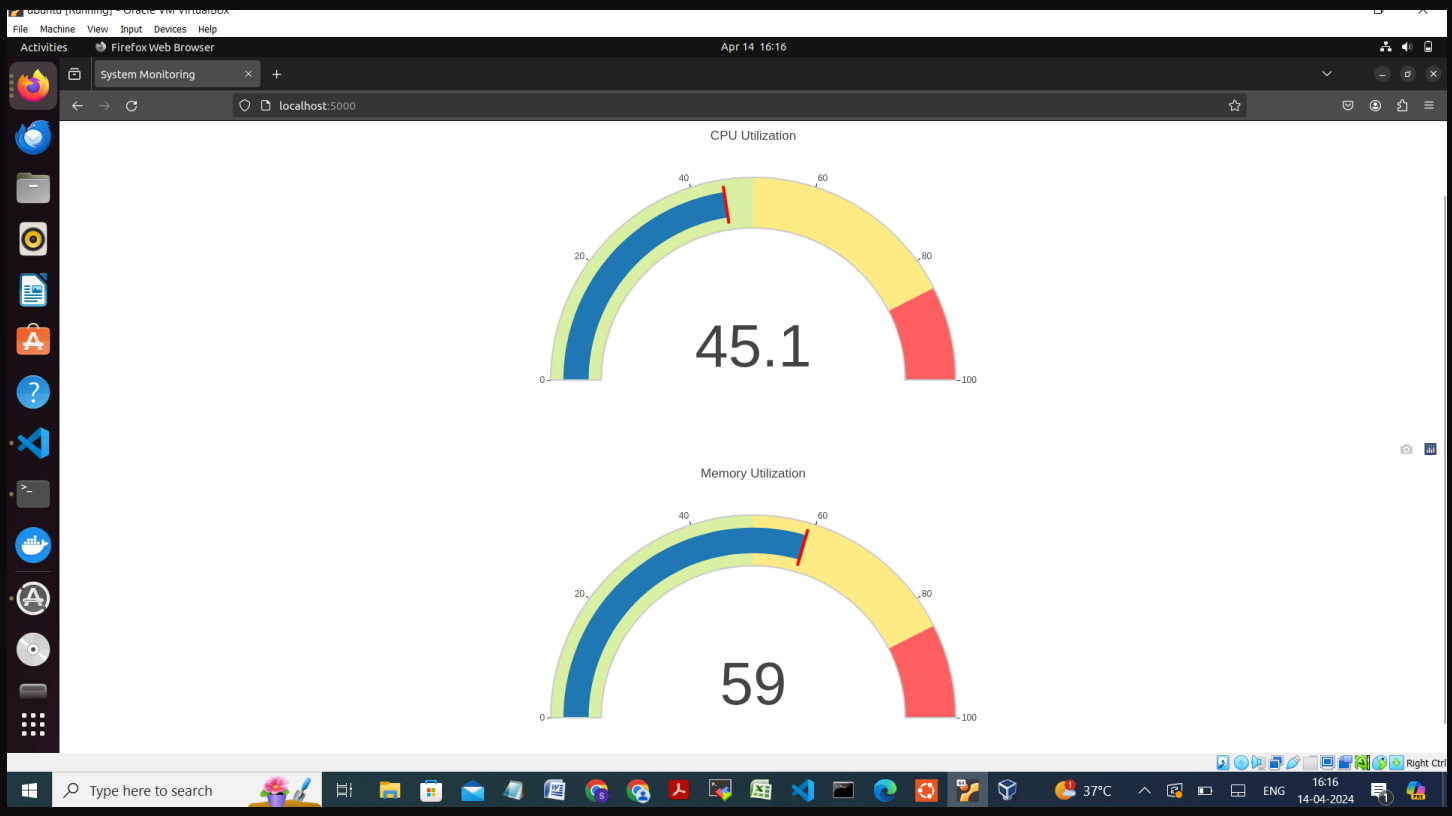
Now to create a container, run below command:

command: docker run –p 5000:5000 d39bbeb4f1e8 - starts a Docker container from the image with ID d39bbeb4f1e8 and maps port 5000 on the host machine to port 5000 inside the container.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

sakeena@ubuntu:~/cloud-native-monitoring-app$ docker run -p 5000:5000 d39bbeb4f1e8
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
```

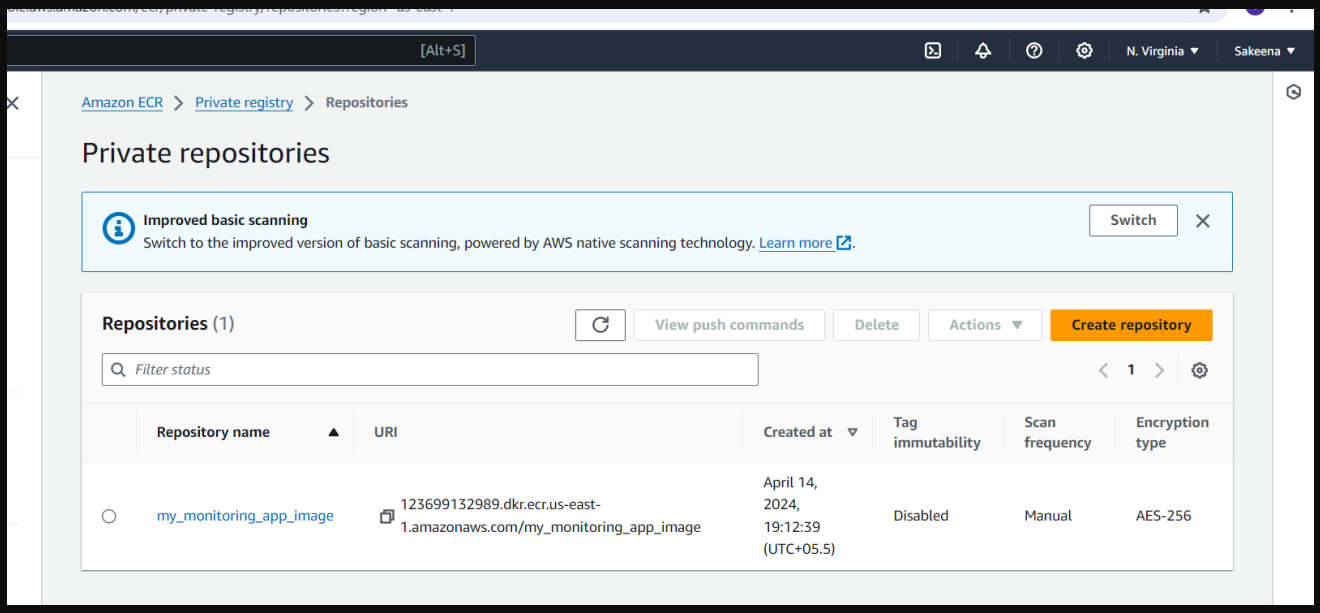
You can access in browser via localhost:5000, which means your application running fine on docker container.



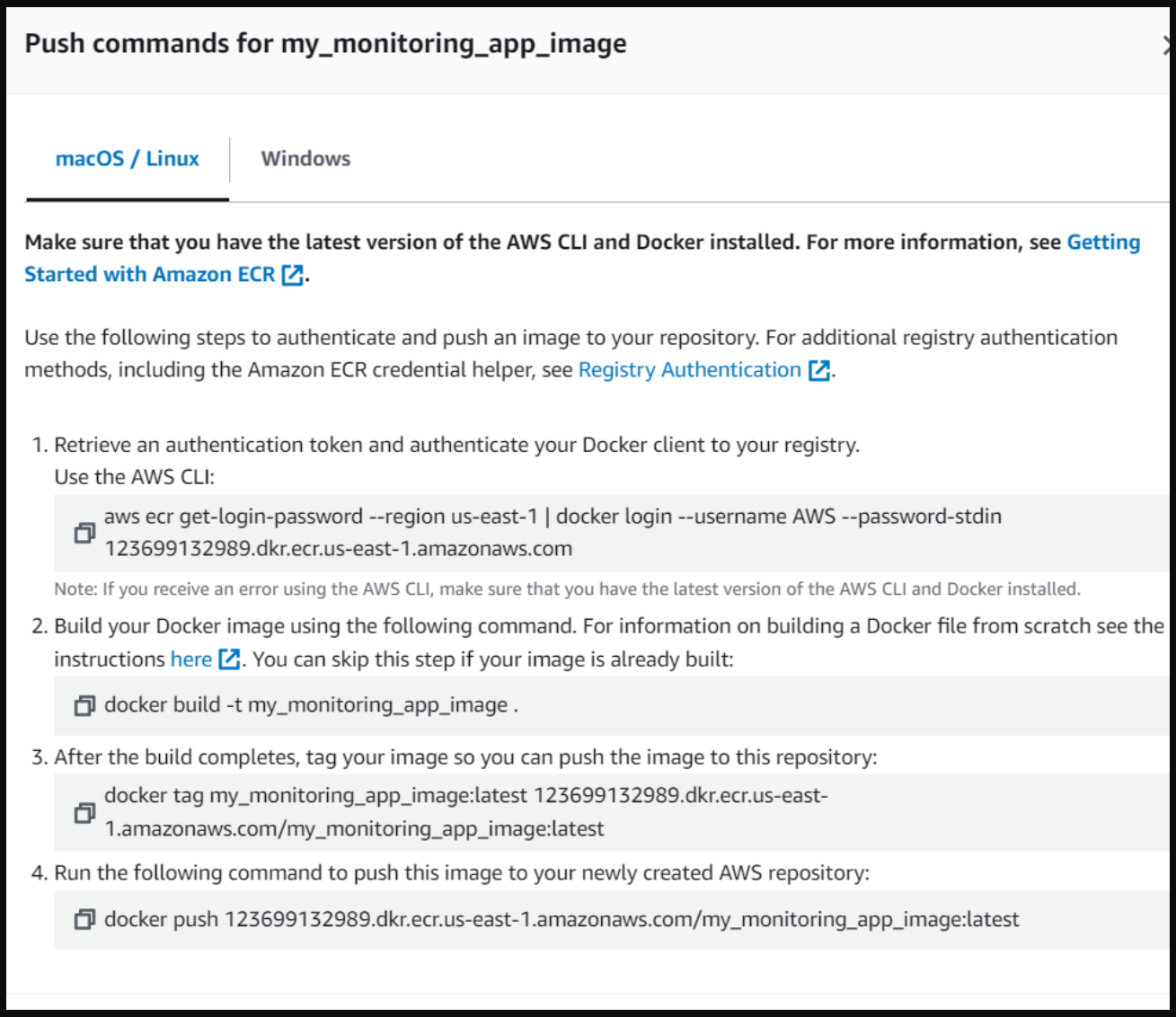
Step7: To push the image to ecr registry
To create a repo using python script (here refer to ecr.py script in git repo)
command: python3 ecr.py

This will create an ecr repo in aws using python

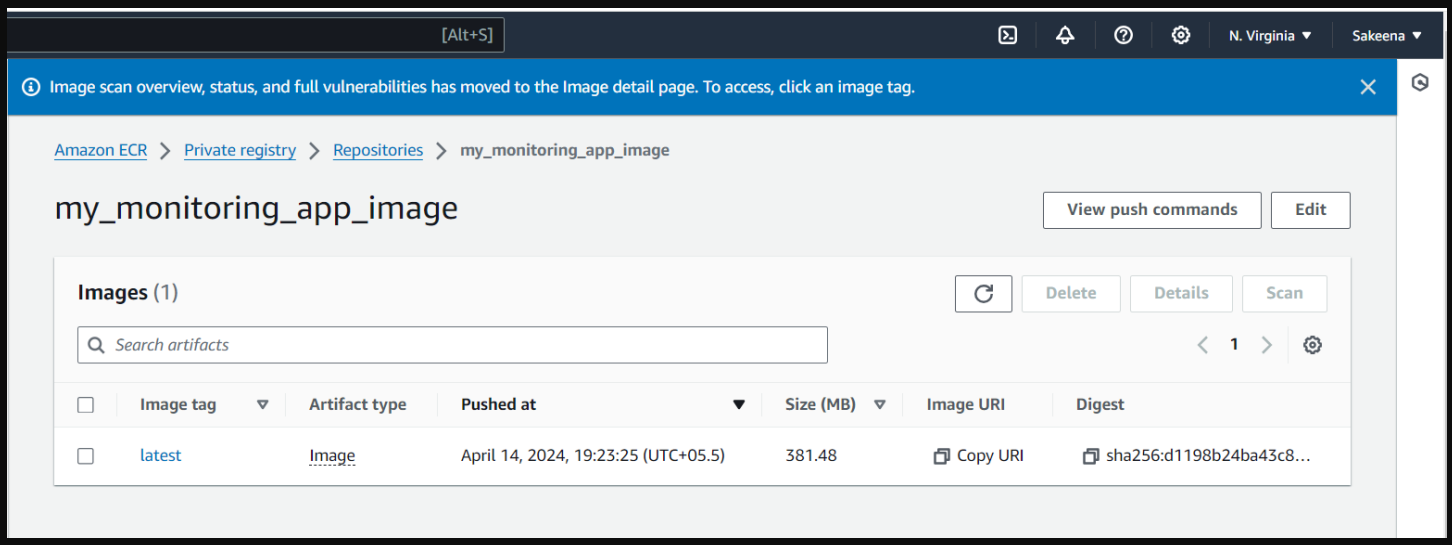
```
4. 44.203.181.170 (ubuntu)
ubuntu@ip-172-31-80-235:~/cloud-native-monitoring-app$ python3 ecr.py
123699132989.dkr.ecr.us-east-1.amazonaws.com/my_monitoring_app_image
ubuntu@ip-172-31-80-235:~/cloud-native-monitoring-app$
```



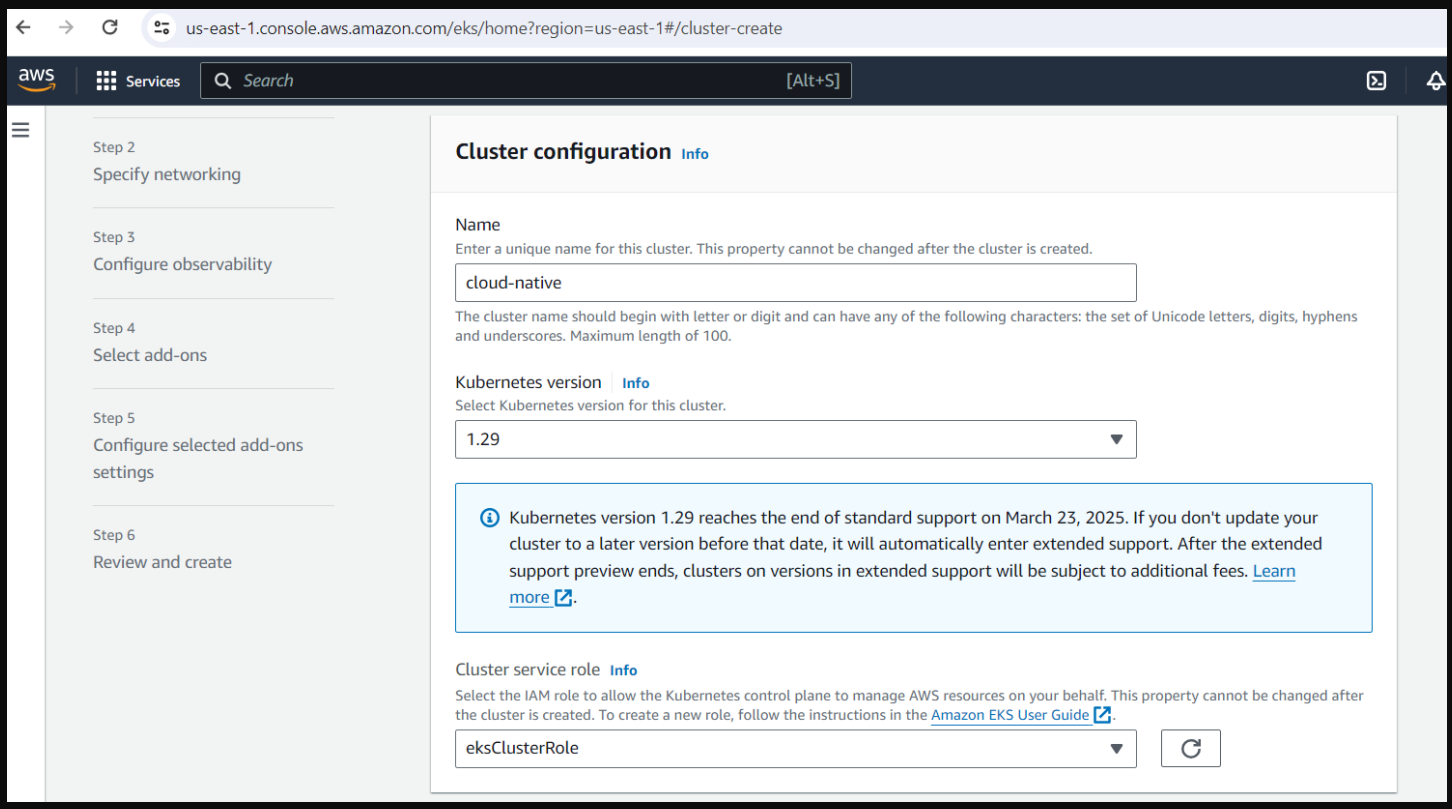
Step8: To push the image in ecr in the created repository .
click on view push commands and execute all commands one by one to push the image in to ecr.



Now you can view the image has been pushed in to the created repo.



Step8: To deploy in to kubernetes cluster,
Navigate to EKS dashboard -> create cluster



Offer the name, default Kubernetes version, cluster service role, default VPC, preferred subnets, a valid security group, and an IPV4 cluster IP, which can be set to either public or private based on your needs. Supplying these particulars is adequate for cluster creation.

Networking Info

IP address family and service IP address range cannot be changed after cluster creation.

VPC Info

Select a VPC to use for your EKS cluster resources.To create a new VPC, go to the VPC console.

vpc-021aa4b98c93ab940 | Default

Subnets Info

Choose the subnets in your VPC where the control plane may place elastic network interfaces (ENIs) to facilitate communication with your cluster. To create a new subnet, go to the corresponding page in the VPC console.

Select subnets

subnet-0ba3952d6599124d1
us-east-1a 172.31.0.0/20

subnet-054fcd10acbacc794
us-east-1d 172.31.32.0/20

subnet-026fb99d2d96d5f14
us-east-1b 172.31.80.0/20

subnet-0db19e7b80562e830
us-east-1c 172.31.16.0/20

subnet-06de4c23a0116763b
us-east-1f 172.31.64.0/20

Clear selected subnets

Security groups Info

Choose the security groups to apply to the EKS-managed Elastic Network Interfaces that are created in your control plane subnets. To create a new security group, go to the corresponding page in the VPC console.

Select security groups

sg-0019fb4f2f78693fa | launch-wizard-5
launch-wizard-5 created 2024-03-31T03:45:56.731Z

Choose cluster IP address family Info

Specify the IP address type for pods and services in your cluster.

- (●) IPv4
- ( ) IPv6

us-east-1.console.aws.amazon.com/eks/home?region=us-east-1#/clusters/cloud-native?selectedTab=cluster-overview-tab

Search [Alt+S] N. Virginia Sakeena

Next step: Provision compute capacity for your cluster by adding a Managed node group or creating a Fargate profile.

EKS > Clusters > cloud-native

cloud-native

Delete cluster

Cluster info Info

Status	Kubernetes version Info	Support type	Provider
Active	1.29	Standard support until March 23, 2025	EKS

Once the cluster is active, we need to create node groups on this cluster as follows:

Configure node group Info

A node group is a group of EC2 instances that supply compute capacity to your Amazon EKS cluster. You can add multiple node groups to your cluster.

Node group configuration

These properties cannot be changed after the node group is created.

Name

Assign a unique name for this node group.

nodegroups

The node group name should begin with letter or digit and can have any of the following characters: the set of Unicode letters, digits, hyphens and underscores. Maximum length of 63.

Node IAM role Info

Select the IAM role that will be used by the nodes. To create a new role, go to the IAM console.

AmazonEKSNodeRole

The selected role must not be used by a self-managed node group as this could lead to a service interruption upon managed node group deletion.

Learn more

Node group compute configuration

These properties cannot be changed after the node group is created.

AMI type Info

Select the EKS-optimized Amazon Machine Image for nodes.

Amazon Linux 2 (AL2_x86_64)

Capacity type

Select the capacity purchase option for this node group.

On-Demand

Instance types Info

Select instance types you prefer for this node group.

Enter an instance type

t2.micro

vCPU: 1 vCPU Memory: 1 GiB Network: Low to Moderate Max ENI: 2 Max IPs: 4

Disk size

Select the size of the attached EBS volume for each node.

20 GiB

Node group scaling configuration

Desired size

Set the desired number of nodes that the group should launch with initially.

2 nodes

Desired node size must be greater than or equal to 0

Minimum size

Set the minimum number of nodes that the group can scale in to.

2 nodes

Minimum node size must be greater than or equal to 0

Maximum size

Set the maximum number of nodes that the group can scale out to.

2 nodes

Maximum node size must be greater than or equal to 1 and cannot be lower than the minimum size

Choosing these options will create 2 nodes inside your cluster

Node group update configuration Info

Maximum unavailable

Set the maximum number or percentage of unavailable nodes to be tolerated during the node group version update.

Number
Enter a number

Percentage
Specify a percentage

Value

1 node

Node count must be greater than 0.

If you encounter permission issues when pulling an image from ECR, review the permissions assigned in IAM. Ensure that the necessary policies are attached to grant the appropriate access to ECR resources

Node group network configuration

These properties cannot be changed after the node group is created.

Subnets Info

Specify the subnets in your VPC where your nodes will run.To create a new subnet, go to the corresponding page in the VPC console.

Select subnets

subnet-0ba3952d6599124d1

subnet-054fcd10acbacc794

subnet-026fb99d2d96d5f14

subnet-0db19e7b80562e830

subnet-06de4c23a0116763b

Clear selected subnets

Configure remote access to nodes Info

Step9:

To deploy in to kubernetes cluster, we use 2 concepts
1) kubernetes deployment: Kubernetes Deployment is a way to manage and control how your application runs on Kubernetes. It specifies what container images to use, how many instances of those containers should run, and how to update them over time
2) kubernetes service: Kubernetes Service is a way to make your application accessible to other parts of your Kubernetes cluster or to the outside world

```
7. 3.83.43.249 (ubuntu)    8. 3.83.43.249 (ubuntu)
ubuntu@ip-172-31-80-75:~/cloud-native-monitoring-app$ kubectl get pods -n default
NAME                          READY   STATUS    RESTARTS   AGE
my-flask-app-7dddbf8ddb-4kws5   1/1     Running   0          2m5s
ubuntu@ip-172-31-80-75:~/cloud-native-monitoring-app$ kubectl get deployment -n default
NAME           READY   UP-TO-DATE   AVAILABLE   AGE
my-flask-app   1/1     1            1           2m37s
ubuntu@ip-172-31-80-75:~/cloud-native-monitoring-app$ kubectl get service -n default
NAME               TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
kubernetes         ClusterIP   10.96.0.1        <none>        443/TCP    103m
my-flask-service   ClusterIP   10.107.195.159   <none>        5000/TCP   72m
ubuntu@ip-172-31-80-75:~/cloud-native-monitoring-app$ kubectl port-forward svc/my-flask-service 5000:5000
Forwarding from 127.0.0.1:5000 -> 5000
Forwarding from [::1]:5000 -> 5000
```

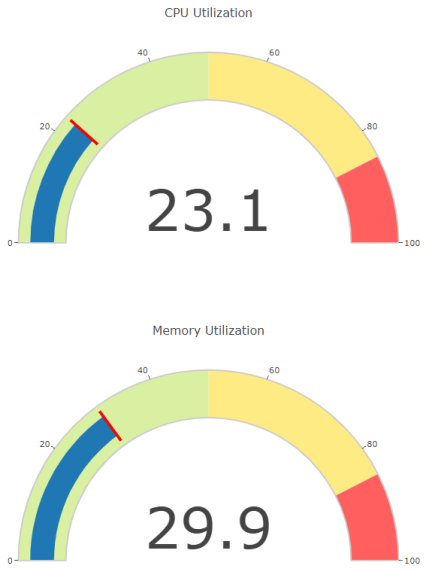
We can also use minikube in case of deploying a single-node kubernetes cluster as follows:

```
7. 3.83.43.249 (ubuntu)    8. 3.83.43.249 (ubuntu)    9. 54.91.205.85 (ubuntu)
ubuntu@ip-172-31-20-218:~/cloud-native-monitoring-app$ kubectl get pods -n default
NAME                          READY   STATUS    RESTARTS   AGE
my-flask-app-7dddbf8ddb-r74fd   1/1     Running   0          29m
ubuntu@ip-172-31-20-218:~/cloud-native-monitoring-app$ kubectl get svc -n default
NAME               TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)    AGE
kubernetes         ClusterIP   10.96.0.1     <none>        443/TCP    36m
my-flask-service   ClusterIP   10.97.26.85   <none>        5000/TCP   30m
ubuntu@ip-172-31-20-218:~/cloud-native-monitoring-app$ minikube service my-flask-service
|-----------|------------------|-------------|--------------|
| NAMESPACE |       NAME       | TARGET PORT |     URL      |
|-----------|------------------|-------------|--------------|
| default   | my-flask-service |             | No node port |
|-----------|------------------|-------------|--------------|
* service default/my-flask-service has no node port
ubuntu@ip-172-31-20-218:~/cloud-native-monitoring-app$ kubectl edit svc my-flask-service
service/my-flask-service edited
```

System Monitoring

Not secure 54.91.205.85:5000

System Monitoring



Type here to search

ENG 11:40 15-04-2024